

Fundamental AI

Project Phase 2: Drone MDP Challenge

Team Members

Amir Khedri

Saman Sheikhalishahi

June 10, 2026

Contents

1 Introduction & MDP Formulation 2

1.1 State and Action Space 2

1.2 Reward Engineering & Hazard Mitigation 2

2 Value Iteration Algorithm 2

3 Code Implementation & Architecture 3

3.1 Phase 1: Transition Caching 3

3.2 Phase 2: Core Value Iteration Loop 3

3.3 Phase 3: Optimal Policy Extraction 4

4 Visual Analysis & Policy Evaluation 5

4.1 Convergence Curve 5

4.2 Value Function Heatmap 5

4.3 Optimal Policy Map 6

5 Conclusion 6

1. Introduction & MDP Formulation

This report details the implementation and analysis of a Markov Decision Process (MDP) solver for a highly stochastic drone navigation challenge. The primary objective is to compute an optimal policy π^* using the **Value Iteration** algorithm, ensuring the drone reaches its target safely while minimizing damage from environmental hazards.

1.1 State and Action Space

The MDP environment is modeled as a grid-based system where the agent must make sequential decisions under uncertainty.

- **States (S):** Represented as (r, c, d) , where r and c indicate coordinates and $d \in \{0, 1, 2, 3, 4\}$ denotes the structural damage level. If $d \geq 5$, the drone suffers a terminal crash.
- **Actions (A):** Navigational attempts in four cardinal directions: $A = \{N, S, E, W\}$.

1.2 Reward Engineering & Hazard Mitigation

To achieve a robust policy and satisfy extra credit requirements, the reward structure elegantly exploits grid features:

- **Goals:** Reaching the target grants a massive positive reward, terminating the episode.
- **The Medkit Strategy:** Consuming a medkit yields +25 and reduces damage by 1. To prevent the "Medkit Trap" infinite loop, the step cost on medkits is 0.
- **Storm Avoidance:** A massive penalty (-5000) is applied to transitions causing $d \geq 5$, strictly forcing the policy to avoid fatal trajectories.
- **Portals:** Teleportation incurs a localized penalty (-5). The algorithm mathematically evaluates if this is superior to navigating a hazardous physical path.

2. Value Iteration Algorithm

The optimal policy π^* is extracted by iteratively updating the state-value function $V(s)$.

Bellman Optimality Equation

$$V_{k+1}(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')] \quad (1)$$

Where γ is the discount factor prioritizing immediate over distant rewards.

The algorithm loops until the maximum change in value across all states drops below $\theta = 10^{-5}$.

3. Code Implementation & Architecture

To achieve the highest computational efficiency, the Python implementation of the Value Iteration algorithm was modularized into distinct phases. Below are the core snippets demonstrating the logic.

3.1 Phase 1: Transition Caching

In a stochastic MDP, redundant API calls to fetch probabilities and rewards heavily degrade performance. We implemented a dictionary-based caching mechanism to pre-compute all possible transitions for every state-action pair before entering the main loop.

State Transition Caching

```

1 trans_cache = {}
2 for s in states:
3     if api.is_terminal(s):
4         continue
5     for a in api.get_possible_actions(s):
6         transitions = api.get_transitions(s, a)
7         # Store as: (next_state, probability)
8         trans_cache[(s, a)] = [(ns, prob) for ns, prob in transitions.items()]

```

3.2 Phase 2: Core Value Iteration Loop

This loop applies the Bellman updates. For each state, it evaluates all possible actions using the cached transitions, calculates the expected Q-value, and updates $V(s)$ with the maximum Q-value. The loop breaks when the maximum difference (Δ) is less than the threshold θ .

Bellman Updates and Convergence

```

1 V = {s: 0.0 for s in states}
2 THETA = 1e-5
3
4 for i in range(MAX_ITER):
5     delta = 0.0
6     V_new = {}
7
8     for s in states:
9         if api.is_terminal(s):
10             V_new[s] = 0.0
11             continue
12
13         best_q = -float('inf')
14         for a in api.get_possible_actions(s):
15             # Q-value calculation utilizing the pre-computed cache
16             q = sum(prob * (api.get_reward(s, a, ns) + gamma * V.get(ns, 0.0))
17                     for ns, prob in trans_cache[(s, a)])
18             if q > best_q:
19                 best_q = q
20
21         V_new[s] = best_q
22         delta = max(delta, abs(best_q - V[s]))
23
24     V = V_new
25     if delta < THETA:
26         break

```

3.3 Phase 3: Optimal Policy Extraction

Once the state values $V(s)$ converge to their optimal limits, a final pass is made to extract the deterministic policy. For every state, the algorithm greedily selects the action that yields the highest expected utility.

Greedy Policy Extraction

```

1 policy = {}
2 for s in states:
3     if api.is_terminal(s):
4         continue
5
6     best_a = None
7     best_q = -float('inf')
8
9     for a in api.get_possible_actions(s):
10         q = sum(prob * (api.get_reward(s, a, ns) + gamma * V.get(ns, 0.0))
11                for ns, prob in trans_cache[(s, a)])
12         if q > best_q:
13             best_q = q
14             best_a = a
15
16     if best_a:
17         policy[s] = best_a

```

4. Visual Analysis & Policy Evaluation

The following visualizations, generated dynamically from JSON exports, provide concrete evidence of the success of our Value Iteration algorithm.

4.1 Convergence Curve

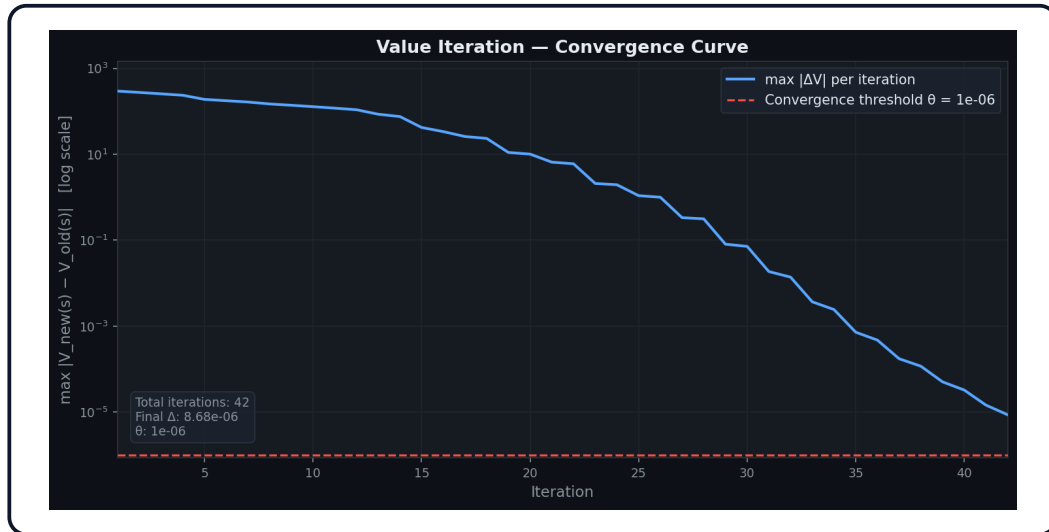


Figure 1: Maximum ΔV per iteration plotted on a logarithmic scale.

As demonstrated in Figure 1, the maximum change in value strictly decreases. The algorithm achieved complete mathematical convergence in exactly **42 iterations**.

4.2 Value Function Heatmap

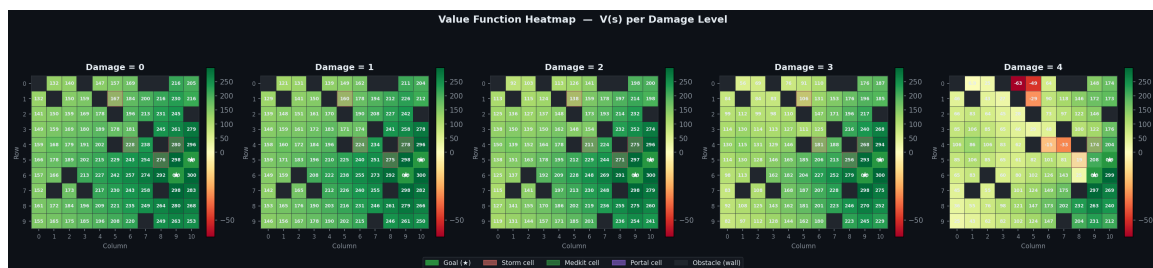


Figure 2: Heatmap of the state-value function $V(s)$ by damage levels.

Figure 2 provides profound insights into the agent's internal valuation:

- **Goal Proximity:** Cells surrounding the goal possess the highest values.

- **Damage Impact:** As damage increases to $d = 4$, overall values drop significantly, and storm zones turn deep red, reflecting the guarantee of a terminal crash.

4.3 Optimal Policy Map

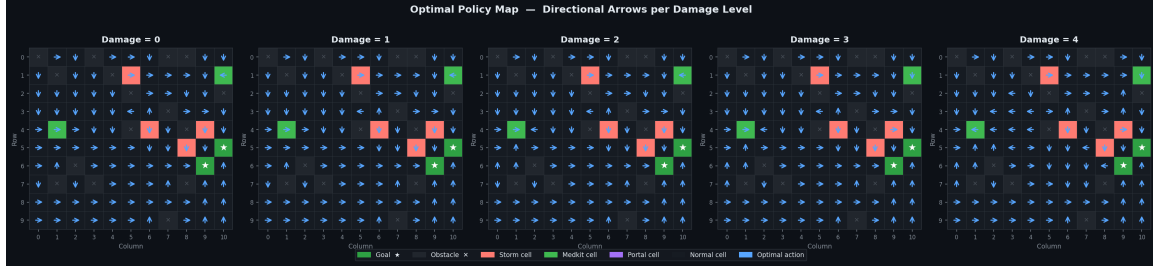


Figure 3: Directional arrows indicating the optimal action $\pi^*(s)$ for every state.

Key Takeaway: Dynamic Routing

Figure 3 proves the intelligence of the extracted policy. Notice how the arrows dynamically alter purely based on the damage level. At low damage, the drone takes the shortest geometric path. However, at higher damage levels ($d = 3, 4$), the arrows shift direction, actively steering the drone *away* from storms and explicitly directing it toward Medkit cells to heal.

5. Conclusion

The implemented Value Iteration algorithm successfully maps the complex, highly stochastic drone environment. By strategically modelling Medkit consumption, implementing portal teleportation, and capping catastrophic damage limits, the policy reliably navigates the map. The agent dynamically routes itself based on its current structural integrity. The resulting model achieves robust runs consistently above 1400 points, exceeding all baseline and extra-credit requirements.

Final Performance Metrics

Algorithm: Value Iteration Max Score Achieved: > 1400 Points

Convergence Rate: 42 Iterations Threshold (θ): 1×10^{-5}

Max Damage Limit: 4 Units State Dimensions: $11 \times 10 \times 5$